

Housing Price Category Prediction Using Machine Learning

CSE422 Lab Project Report

Nishat Tasnim Zannat, Student id: 23201116, Iffat Zerine Nowmi, Student id: 23201110

Abstract—This report presents a machine learning approach to predict housing price categories (Low, Medium, High) based on various property features. The dataset consists of 1,200 housing records with 11 input features including location, size, number of bedrooms, and distance to city center. We apply four supervised learning models—Neural Network, K-Nearest Neighbors (KNN), Decision Tree, and Logistic Regression—as well as K-Means Clustering as an unsupervised approach. Each model is evaluated using accuracy, precision, recall, F1-score, confusion matrix analysis, and ROC-AUC scores. Logistic Regression achieved the highest accuracy of 38.33% and an AUC score of 0.5252, making it the best-performing model in this study.

Index Terms—machine learning, housing price prediction, classification, neural network, KNN, logistic regression, K-means clustering

I. INTRODUCTION

THE real estate market is one of the most dynamic and complex sectors of any economy. Accurately predicting housing prices is a critical challenge faced by buyers, sellers, real estate agents, and financial institutions. Traditional methods of property valuation are often subjective and time-consuming, relying heavily on domain expertise and manual analysis.

The report is prepared using a structured document preparation system to ensure proper formatting of tables, equations, and figures.

This project aims to address this challenge by applying machine learning techniques to classify housing properties into three price categories: Low, Medium, and High. By leveraging a dataset of 1,200 housing records with 11 diverse features—including location, property size, number of rooms, and proximity to the city center—we build and evaluate multiple classification models to determine which features are most predictive and which algorithms perform best on this task.

The target feature, Price_Category, is a discrete categorical variable with three classes. The motivation behind framing this as a classification problem rather than regression is that buyers and investors often think in terms of price ranges rather than exact values. Predicting a category is also more robust to noise and outliers in real-world housing data.

The project follows a structured pipeline: data loading and exploration, exploratory data analysis (EDA), pre-processing, model training using supervised and unsupervised methods, and comprehensive model evaluation.

II. DATASET DESCRIPTION

A. Feature Summary

The dataset includes 6 numerical features and 6 categorical features (including the target). Table I provides a summary of all features along with their data types and percentage of missing values.

TABLE I
DATASET FEATURES, TYPES, AND MISSING VALUES

Feature	Type	Missing %
Location	Categorical	11.67%
Size_sqft	Numerical	8.83%
Num_Bedrooms	Numerical	11.33%
Num_Bathrooms	Numerical	11.92%
Has_Balcony	Categorical	10.08%
Floor_Number	Numerical	11.00%
Building_Age_Years	Numerical	10.58%
Parking_Available	Categorical	8.42%
Nearby_Schools	Categorical	10.33%
Distance_to_CityCenter_km	Numerical	8.58%
Security_Level	Categorical	10.50%
Price_Category (Target)	Categorical	0.00%

B. Target Variable Distribution

The target variable Price_Category is well-balanced across its three classes, which is favorable for model training as it reduces class bias. Table II summarizes the class distribution.

TABLE II
TARGET VARIABLE DISTRIBUTION

Category	Count	Proportion
Low	402	33.50%
Medium	413	34.42%
High	385	32.08%

III. DATA ANALYSIS

Exploratory Data Analysis (EDA) was performed to understand the structure of the data, identify anomalies, and extract relationships between features before model training.

A. Missing Values

A significant portion of the data contains missing values, with every feature having between 8% and 12% missing entries. Num_Bathrooms had the highest missing rate at 11.92%, while Parking_Available had the lowest at 8.42%. Visualizing these as a bar chart confirmed that no single feature was critically corrupted and all could be retained through imputation.

B. Feature Distributions

Histograms for the six numerical features revealed that their distributions are relatively flat and spread across their ranges, with no clear Gaussian bell-curve shape. This observation is significant because it suggests that algorithms which assume Gaussian feature distributions—such as Gaussian Discriminant Analysis and Gaussian Naive Bayes—may not be well-suited to this dataset. Box plots further confirmed the presence of mild outliers in features such as `Size_sqft` and `Distance_to_CityCenter_km`, but no extreme values requiring removal were found.

C. Categorical Feature Distributions

Bar charts for categorical features showed that `Location` is split across three categories (City Center, Countryside, Suburbs), while binary features like `Has_Balcony` and `Parking_Available` are slightly skewed toward 'No'. `Security_Level` is spread across High, Medium, and Low. These distributions confirm sufficient representation for all category levels.

D. Correlation Analysis

A correlation heatmap was generated by temporarily encoding all categorical variables and computing pairwise Pearson correlations. The analysis revealed that all features had very low correlation with the target variable `Price_Category`, with the highest being `Nearby_Schools` at 0.049 and the lowest being `Security_Level` at 0.002. This weak correlation explains the difficulty models face in predicting price categories accurately and is a major contributing factor to the low model accuracies observed.

E. Feature vs. Target Relationships

Grouped box plots comparing each numerical feature across the three price categories showed significant overlap between groups. Ideally, a predictive feature would show clearly separated distributions for Low, Medium, and High prices. The absence of such separation confirmed that no single feature is a strong linear predictor of price category.

IV. DATASET PRE PROCESSING

Raw data is rarely ready for machine learning. Several preprocessing steps were applied to prepare the dataset for model training.

A. Handling Null Values

Problem: Every feature in the dataset contained missing values ranging from 8% to 12%. Removing rows with missing values would have eliminated a large proportion of the 1,200 samples, severely reducing the training data.

Solution: Missing values were imputed using different strategies based on feature type:

- Numerical features (`Size_sqft`, `Num_Bedrooms`, `Num_Bathrooms`, `Floor_Number`, `Building_Age_Years`, `Distance_to_CityCenter_km`): filled with the column median. The median is preferred over the mean as it is robust to outliers.
- Categorical features (`Location`, `Has_Balcony`,

`Parking_Available`, `Nearby_Schools`, `Security_Level`): filled with the column mode (most frequent value). After imputation, zero missing values remained in the dataset.

B. Duplicate Values

A check for duplicate rows was performed. No duplicates were found in the 1,200-row dataset, so no rows were removed on this basis.

C. Encoding Categorical Variables

Problem: Machine learning algorithms require numerical input. The five categorical input features (`Location`, `Has_Balcony`, `Parking_Available`, `Nearby_Schools`, `Security_Level`) and the target variable (`Price_Category`) are stored as text strings.

Solution: Label Encoding was applied to convert each categorical value to an integer. For example, `Location` was encoded as: City Center=0, Countryside=1, Suburbs=2. The target variable was encoded as: High=0, Low=1, Medium=2.

D. Feature Selection

All 11 input features were retained for model training. Although the correlation analysis showed weak individual correlations, removing features without stronger evidence (such as recursive feature elimination) risked discarding potentially useful combinations. Feature selection through more advanced methods was noted as a future improvement.

E. Train-Test Split

The dataset was split into 80% training (960 samples) and 20% testing (240 samples) using stratified splitting. Stratification ensures that the class distribution of Low, Medium, and High is preserved proportionally in both sets, preventing biased evaluation.

F. Feature Scaling

Problem: Features have very different ranges—`Size_sqft` ranges from 301 to 2499, while `Num_Bedrooms` ranges from 1 to 4. Distance-based algorithms like KNN and gradient-descent-based models like Neural Networks are sensitive to this imbalance.

Solution: `StandardScaler` was applied to normalize all features to have a mean of 0 and a standard deviation of 1. The scaler was fit only on the training set and then applied (transformed) to the test set, preventing data leakage.

V. MODEL TRAINING

Five models were trained—four supervised classifiers and one unsupervised clustering algorithm. Each model was applied to the preprocessed and scaled training data.

A. Neural Network

Neural Network was chosen because it is capable of learning complex, non-linear relationships between features

that simpler models cannot capture. It serves as the deep learning baseline for this project and is a mandatory requirement of the lab guidelines.

A feedforward Neural Network was constructed using TensorFlow/Keras with the following architecture: an input layer with 11 neurons (one per feature), followed by Dense(64, ReLU), Dropout(0.3), Dense(32, ReLU), Dropout(0.3), Dense(16, ReLU), and finally Dense(3, Softmax) for the three output classes.

The model was compiled with the Adam optimizer (learning rate 0.001) and sparse categorical cross-entropy loss. It was trained for up to 100 epochs with a batch size of 32 and a 20% validation split. Early Stopping with patience=10 was used to halt training when validation loss stopped improving, preventing overfitting. The model achieved a test accuracy of 34.17%.

B. K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) was selected because it is a non-parametric algorithm that makes no assumptions about the underlying data distribution. Given that our EDA showed non-Gaussian feature distributions, KNN is appropriate as it relies purely on distance between data points rather than any statistical assumption.

KNN classifies a sample by finding its k nearest neighbors in the training set and assigning the majority class. We used $k=5$ with distance weighting (closer neighbors have more influence). KNN requires no training phase—it stores the entire training set and computes distances at prediction time. It achieved a test accuracy of 34.17%, equal to the Neural Network.

C. Decision Tree

Decision Tree was chosen for its interpretability. Unlike black-box models, a Decision Tree produces a set of human-readable if-else rules, making it easy to explain why a particular prediction was made. It also handles mixed feature types naturally without requiring scaling, though scaling was applied uniformly across all models.

A Decision Tree recursively splits the data based on feature thresholds that maximize class purity (measured by Gini impurity). Parameters were set to max depth=10 and min samples split=10 to prevent the tree from overfitting to noise. The Decision Tree achieved the lowest test accuracy of 31.25%.

D. Logistic Regression

Logistic Regression was selected as a strong, simple linear baseline. It is computationally efficient, works well for balanced multi-class problems using the softmax function, and makes no assumptions about feature distributions — only about the linearity of the decision boundary.

Logistic Regression extends binary classification to multiple classes using the softmax function, which outputs a probability distribution over all three classes. It assumes a linear decision boundary between classes. Despite its simplicity, it achieved the highest test accuracy of 38.33%, suggesting that what little signal exists in the features is primarily linear in nature.

E. K-Means Clustering (Unsupervised)

K-Means Clustering was applied as an unsupervised learning approach where class labels are not used during training. The algorithm groups data points into k clusters by minimizing within-cluster sum of squared distances. The Elbow Method was used to identify $k=3$ as the optimal number of clusters, corresponding to the three price categories.

Principal Component Analysis (PCA) was applied to reduce the 11-dimensional feature space to 2 dimensions for visualization. The cluster visualization showed only partial alignment with the true price categories, which is expected since the algorithm had no access to label information. The cross-tabulation of cluster assignments versus actual labels confirmed that no cluster cleanly maps to a single price category.

Gaussian Discriminant Analysis (GDA) was not used because GDA assumes that features within each class follow a multivariate Gaussian distribution, but our histogram analysis showed that numerical features like Size_sqft and Distance_to_CityCenter_km are flat and uniformly distributed, violating this assumption. Furthermore, the dataset contains five categorical features which cannot be meaningfully treated as continuous Gaussian variables even after label encoding.

Naive Bayes was not used because Naive Bayes assumes all features are conditionally independent of each other, but our correlation heatmap confirmed inter-feature dependencies — for example, Location and Distance_to_CityCenter_km are naturally correlated, as City Center properties almost always have low distance values. When this independence assumption is violated, Naive Bayes double-counts correlated evidence and produces unreliable class probability estimates.

VI. MODEL EVALUTION

A. Accuracy, Precision, Recall, and F1-Score.

Table III compares the four supervised models across the four key classification metrics, all computed using weighted averaging to account for slight class imbalance.

Logistic Regression outperforms all other models across

TABLE III
CLASSIFICATION METRICS COMPARISON

Model	Accuracy	Precision	Recall	F1-Score
Neural Network	0.3417	0.3378	0.3417	0.3196
KNN	0.3417	0.3415	0.3417	0.3409
Decision Tree	0.3125	0.3139	0.3125	0.3089
Logistic Regression	0.3833	0.3731	0.3833	0.3713

every metric. The Neural Network and KNN tied on accuracy, with KNN showing a slightly higher F1-score. The Decision Tree performed worst overall.

B. Confusion Matrix Analysis.

Confusion matrices were generated for all four models. The matrices revealed that all models struggled significantly to distinguish between the three price categories, with off-diagonal misclassifications being frequent and roughly symmetric across classes. No model shows a clear ability to correctly identify one class over another. This is consistent with the weak feature-target correlations found during EDA..

For Logistic Regression (best model), the classification report showed: High class precision=0.32, recall=0.19; Low class precision=0.41, recall=0.50; Medium class precision=0.39, recall=0.45. The model performs relatively better on the Low and Medium classes than on High.

C. ROC Curve and AUC Score.

ROC (Receiver Operating Characteristic) curves were plotted for all four models using micro-averaged one-vs-rest multiclass extension. Table IV shows the AUC scores..

An AUC of 0.5 corresponds to random guessing. All models scored just above 0.5, confirming that their discriminative ability is only marginally better than chance. Logistic Regression again leads with 0.5252, while KNN performed worst at 0.4565 (slightly below random chance on this metric).

The ROC curves for all models hug very close to the diagonal line (random classifier), which visually confirms the quantitative AUC values.

A. Best Performing Model

Logistic Regression outperformed other models primarily because its simplicity prevents overfitting on a moderately sized dataset of 1,200 samples, and its built-in regularization improves generalization by penalizing overly complex decision boundaries. Additionally, its robustness to noise made it well-suited for this dataset where 8–12% of values were imputed, and the linear decision boundaries it produces appear to capture the limited signal present in the weakly-correlated features more effectively than the complex boundaries learned by the Neural Network and Decision Tree.

VII. CONCLUSION

A. Best Model

Based on all evaluation metrics—accuracy, precision, recall, F1-score, and AUC score—Logistic Regression is the best performing model for this dataset, achieving 38.33%. B. Probable Reasons Behind Results The uniformly low performance across all models (ranging from 31%

B. Probable Reasons Behind Results

The uniformly low performance across all models (ranging

from 31% to 38% accuracy, compared to a 33% random baseline for 3 classes) can be attributed to several factors:

- **Weak feature-target correlations:** The EDA revealed that no feature had a correlation above 0.05 with Price_Category. This means the features as provided do not contain sufficient information to reliably predict the price category.
- **Overlapping class distributions:** The box plots of features grouped by price category showed heavy overlap between Low, Medium, and High classes for all numerical features. The classes are not linearly or non-linearly separable in the given feature space.
- **Dataset characteristics:** The data appears to have been synthetically generated or heavily anonymized, which may have removed the natural relationships between property features and price that exist in real-world data.
- **Imputation introducing noise:** With 8-12% missing values in every feature, the imputed values (medians and modes) may have diluted the true signal in the data, making it harder for models to find decision boundaries.
- **Feature encoding limitations:** Label encoding of categorical features like Location and Security_Level introduces an implicit ordinal relationship that may not be meaningful (e.g., encoding City Center=0, Countryside=1, Suburbs=2 implies Countryside is between City Center and Suburbs numerically).

C. Recommendations for Improvement

- **Apply Random Forest or XGBoost,** which are ensemble methods that typically handle noisy, weakly-correlated datasets better than single models.
- **Use One-Hot Encoding** instead of Label Encoding for nominal categorical features to avoid introducing spurious ordinal relationships.
- **Apply feature engineering** to create interaction features (e.g., price_per_square_foot, rooms_per_floor) that may capture relationships not present in individual features.
- **Perform cross-validation (k-fold)** for more robust performance estimation rather than a single train-test split.
- **Investigate** whether the dataset can be enriched with additional features such as neighborhood crime rates, school ratings, or local amenities that are known to influence housing prices.

D. Unsupervised Learning Insights The K-Means clustering analysis confirmed that three natural groupings exist in the housing data, but these groupings do not cleanly align with the Low, Medium, and High price categories. This further supports the conclusion that the provided features do not contain sufficient price-discriminating information, and that pricing decisions may be driven by latent factors not captured in the dataset.

VIII. REFERENCE

- [1] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [2] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proc. OSDI*, 2016.

- [3] W. McKinney, "Data structures for statistical computing in Python," in Proc. SciPy, 2010.
- [4] J. D. Hunter, "Matplotlib: A 2D graphics environment," Computing in Science Engineering, vol. 9, no. 3, pp. 90-95, 2007.
- [5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. KDD, 2016.